CRUD v4 Blueprint

⚠ MANDATORY — READ BEFORE WRITING ANY CODE

> **This blueprint is the single source of truth for all React feature development.** Every page, every component, every hook, every service — built from now on — follows this document exactly. There are no exceptions and no shortcuts.

This applies to:

- All new features
- All modifications to existing features
- All AI-assisted code generation (Cascade/Copilot/etc.)
- All code reviews — non-compliant code must not be merged

Deviation Rule — STRICT AND NON-NEGOTIABLE

> **If any implementation request would require deviating from any rule in this blueprint:**
>
> 1. **STOP.** Do not implement the deviation silently.
> 2. **IDENTIFY** the specific rule being broken (e.g. "This would violate §5 Component Data Ownership — prop drilling query results").
> 3. **EXPLAIN** why the deviation is being considered and what the alternative would cost.
> 4. **GET EXPLICIT WRITTEN APPROVAL** from the team lead / user before proceeding.
> 5. **DOCUMENT** the approved deviation with a comment in the code and a note in this file.
>
> **Silent deviations are never acceptable — even if the alternative seems simpler, faster, or "just this once". The AI assistant must flag deviations honestly even if the user is the one requesting them.**

What counts as a deviation?

Any of the following require explicit approval:

Situation	Why it's a deviation
Passing query result data as props to a component that can self-fetch	Violates §5 Component Data Ownership
Fetching data inside the page component directly	Violates §2.9 Page = thin orchestrator
Skipping `zodParse` on a read endpoint	Violates §2.3 + §7 OWASP A08
Using `useState` for modal open + editCtx together	Violates §2.8 — must use `useReducer`
Not wrapping a Shadcn input in `<Controller>`	Violates §2.12 Forms rule
Building a custom button/input/modal instead of using Shadcn	Violates §9 Mobile/Shadcn rule
Omitting `aria-label` on an icon-only button	Violates §8 WCAG
Omitting `staleTime` on a reference data query	Violates §2.4 React Query rules
Omitting `useCallback` on a handler passed to a child	Violates §5 Performance rules

Situation	Why it's a deviation
Duplicating `ActionItem`/`ActionsConfig`/`renderActionsCell` in a feature	Violates §6 DRY — import from `@/components/layouts/v1/TableActions`
One-click delete without `AlertDialog` confirmation	Violates §7 OWASP A04 + §8 WCAG

Table of Contents

1. Folder Structure
2. File-by-File Internals
    - 2.1 Services/feature.dtos.ts — API Contracts
    - 2.2 Services/feature.types.ts — UI Domain Types
    - 2.3 Services/feature.services.ts — HTTP Calls
    - 2.4 Services/feature.queries.ts — React Query Hooks
    - 2.5 helpers/mapItemDtoToRow.ts — DTO → Row Mapper
    - 2.6 helpers/buildEditContext.ts — DTO → Edit Context
    - 2.7 columns/itemColumns.tsx — Column Definitions
    - 2.8 hooks/useFeaturePage.ts — Page Hook
    - 2.9 Feature.page.tsx — Orchestrator
    - 2.10 components/ItemTable.tsx — Data Table
    - 2.11 components/ItemModal.tsx — Create/Edit Dialog
    - 2.12 components/ItemForm.tsx — Form Fields
    - 2.13 components/DeleteConfirmModal.tsx — Delete Confirmation
    - 2.14 components/StatusPlaceholder.tsx — Status Display
    - 2.15 Feature.module.tsx — Route Entry Point
3. Data Flow Diagram
4. React Hooks & Patterns — Where Each Is Used
5. Rules & Conventions
6. Architecture Principles
7. Security — OWASP Standards
8. Accessibility — WCAG 2.1 AA
9. Mobile Responsiveness
10. Production Readiness Scorecard
11. Junior Developer Quick-Start Guide

1. Folder Structure

```
src/features/[feature-name]/
├── Services/                    ← API layer (no React, no UI)
│   ├── [feature].dtos.ts        ← Zod schemas + inferred types (API contracts)
│   ├── [feature].types.ts       ← UI-facing types + form Zod schema
│   ├── [feature].services.ts    ← HTTP calls (one function per endpoint)
│   └── [feature].queries.ts     ← React Query hooks (one hook per operation)
├── helpers/                     ← Pure functions (no React, no side effects)
│   ├── mapItemDtoToRow.ts       ← DTO → table row mapper + ItemRow type
│   └── buildEditContext.ts      ← DTO → edit modal context
├── columns/                     ← Table column config (declarative)
│   └── itemColumns.tsx          ← Column defs + visibility + filterable columns +
actions config
```

```
├── hooks/                          ← Custom React hooks
│   └── use[Feature]Page.ts         ← All state + handlers + derived data
├── components/                     ← React components (rendering only)
│   ├── ItemTable.tsx               ← Table with search, filters, pagination, row selection
│   ├── ItemModal.tsx               ← Create/Edit dialog (FormProvider wrapper)
│   ├── ItemForm.tsx                ← Form fields (reads from FormProvider)
│   ├── DeleteConfirmModal.tsx      ← Delete confirmation dialog
│   └── StatusPlaceholder.tsx       ← Empty/error/loading placeholder
├── [Feature].module.tsx            ← Route entry point (lazy + Suspense)
└── [Feature].page.tsx              ← Orchestrator (thin — just wiring)
```

---

2. File-by-File Internals

2.1 `Services/[feature].dtos.ts` — API Contracts

Purpose: Zod schemas that mirror the backend exactly. Types inferred from schemas.

Item	Kind	Description
`ItemDtoSchema`	`z.object(...)`	Shape of one item from the list API
`GetItemsResponseSchema`	`z.array(ItemDtoSchema)`	Response wrapper for list endpoint
`ItemDto`	`type` (inferred)	`z.infer<typeof ItemDtoSchema>`
`GetItemsResponse`	`type` (inferred)	`z.infer<typeof GetItemsResponseSchema>`
`CategoryDtoSchema`	`z.object(...)`	Shape of a dropdown option from API
`GetCategoriesResponseSchema`	`z.object(...)`	Response wrapper for categories
`CategoryDto`	`type` (inferred)	Single category
`GetCategoriesResponse`	`type` (inferred)	Full categories response
`YearDataSchema`	`z.object(...)`	Shape of a year/filter option
`GetYearsRetrieveResponseSchema`	`z.object(...)`	Response wrapper for years/filters
`YearData`	`type` (inferred)	Single year
`GetYearsRetrieveResponse`	`type` (inferred)	Full years response
`CreateItemRequest`	`type` (plain)	POST body for create
`CreateItemResponse`	`type` (plain)	Response from create
`UpdateItemRequest`	`type` (plain)	POST body for update
`UpdateItemResponse`	`type` (plain)	Response from update
`DeleteItemRequest`	`type` (plain)	POST body for delete

Rule: Zod schemas for **responses** (validated at runtime via `zodParse`). Plain types for **requests** (constructed by our code — no need to validate our own output).

Example:

```
import { z } from "zod";

export const ItemDtoSchema = z.object({
  StudentName: z.string(),
  StudentNumber: z.number(),
  CurrentYear: z.string(),
  // ... all fields 1:1 with backend
});

export const GetItemsResponseSchema = z.array(ItemDtoSchema);

export type GetItemsResponse = z.infer<typeof GetItemsResponseSchema>;
export type ItemDto = z.infer<typeof ItemDtoSchema>;

export type CreateItemRequest = {
  studentIds: number[];
  activityId: number;
  // ... fields for POST body
};
```

2.2 `Services/[feature].types.ts` — UI Domain Types

Purpose: Types used by the UI — form schema, edit context. Separate from API DTOs.

Item	Kind	Description
`ItemEditContext`	`type`	Pre-populates the modal (`mode: "create" | "update"`, entity fields, defaults)
`ItemFormSchema`	`z.object(...)`	Zod schema for form validation (`min`, `date`, `refine` rules)
`ItemFormValues`	`type` (inferred)	`z.infer<typeof ItemFormSchema>` — what `useForm` manages

Rule: `ItemFormValues` is always inferred from `ItemFormSchema` — never defined manually. This guarantees the type and validation rules can never drift apart.

Example:

```
import { z } from "zod";

export type ItemEditContext = {
  mode: "create" | "update";
  studentActivityId?: number;
  studentId: number;
  activityId: number;
  completedDate: Date;
  notes: string;
  restrictedNotes: boolean;
};

export const ItemFormSchema = z.object({
  activityId: z.string({ required_error: "Activity is required" }).min(1, "Activity is
required"),
  completedDate: z.date({ required_error: "Date is required" }).refine((d) => d <= new
```

```
  Date(), "Date cannot be in the future"),
    completedBy: z.number({ required_error: "Completed By is required" }).min(1, "Completed
By is required"),
    notes: z.string().default(""),
    restrictedNotes: z.boolean().default(false),
});

export type ItemFormValues = z.infer<typeof ItemFormSchema>;
```

2.3 `Services/[feature].services.ts` — HTTP Calls

Purpose: One function per API endpoint. No business logic, no React.

Item	Kind	Description
`GetItemsParams`	`type`	Server filter params (e.g., `{ year?: string }`)
`getItems(params?)`	`async function`	GET list → `zodParse(schema, response)`
`getCategories()`	`async function`	GET dropdown data → `zodParse(schema, response)`
`getYears()`	`async function`	POST years → `zodParse(schema, response)`
`createItem(payload)`	`async function`	POST create → return `response.data`
`updateItem(payload)`	`async function`	POST update → return `response.data`
`deleteItem(payload)`	`async function`	POST delete → no return

Rule: Read endpoints use `zodParse` for runtime validation. Write endpoints return `response.data` directly.

Example:

```
import apiClient from "@/features/common/API/apiClient";
import { zodParse } from "@/utils/zodParse";
import { GetItemsResponseSchema } from "./[feature].dtos";
import type { GetItemsResponse, CreateItemRequest, CreateItemResponse } from
"./[feature].dtos";

export type GetItemsParams = { year?: string };

export const getItems = async (params?: GetItemsParams): Promise<GetItemsResponse> => {
  const response = await apiClient.get("/api/[endpoint]", { params });
  return zodParse(GetItemsResponseSchema, response.data, "getItems");
};

export const createItem = async (payload: CreateItemRequest): Promise<CreateItemResponse>
=> {
  const response = await apiClient.post<CreateItemResponse>("/api/[endpoint]", payload);
  return response.data;
};
```

2.4 `Services/[feature].queries.ts` — React Query Hooks

Purpose: One hook per operation. Handles caching, invalidation, toasts. Also the correct place for URL param overrides via `select`.

Item	Kind	Description
`QUERY_KEYS`	`const` (`as const`)	`{ items: [...], categories: [...], years: [...] }` — single source of truth for cache keys
`useGetItems(params?)`	`useQuery` hook	`enabled: !!params` (search-button pattern — no fetch until user clicks Search)
`useGetCategories()`	`useQuery` hook	`staleTime: 5min` (rarely changes)
`useGetYears()`	`useQuery` hook	`staleTime: 10min` (rarely changes)
`useCreateItem()`	`useMutation` hook	`onSuccess`: toast + `invalidateQueries(items)`
`useUpdateItem()`	`useMutation` hook	`onSuccess`: toast + `invalidateQueries(items)`
`useDeleteItem(options?)`	`useMutation` hook	`onSuccess`: toast + `invalidateQueries(items)` + `options.onDeleted?.()`

Rules:

- `QUERY_KEYS` uses `as const` for type-safe, readonly literal keys.
- List query uses `enabled: !!params` so data is only fetched after the user clicks Search.
- All mutations invalidate the list query on success (cache busting).
- All mutations show `toast.success` / `toast.error`.
- **URL param overrides belong in the query layer** using React Query's `select` option — NOT in the page hook, NOT as component props. Every consumer of the query automatically gets the overridden values with zero prop drilling.

URL Param Override Pattern (mandatory when query results need to be overridable via URL):

```
import { useSearchParams } from "react-router";

// Pure helper functions — defined OUTSIDE the hook (no React, testable)
const parseBoolParam = (v: string | null): boolean | undefined => {
  if (v === "true") return true;
  if (v === "false") return false;
  return undefined;
};

const VALID_NAME_FORMATS = ["fullName", "firstInitial", "lastOnly", "firstOnly"] as
const;
const parseEnumParam = <T extends string>(v: string | null, valid: readonly T[]): T |
undefined =>
  v && valid.includes(v as T) ? (v as T) : undefined;

export const useGetSettingsQuery = () => {
  const [searchParams] = useSearchParams();
  return useQuery<FeatureSettings, AxiosError>({
```

```
    queryKey: QUERY_KEYS.settings,
    queryFn: FeatureServices.getSettings,
    staleTime: STALE_TIME,
    select: (data): FeatureSettings => ({
      showWidget:  parseBoolParam(searchParams.get("showWidget"))  ?? data.showWidget,
      nameFormat:  parseEnumParam(searchParams.get("nameFormat"), VALID_NAME_FORMATS) ??
data.nameFormat,
    }),
  });
};
```

Why `select` and not `useState` / prop drilling:

- `select` runs after the API response arrives — it transforms data at the query boundary.
- All components that call the same query hook automatically receive the URL-overridden values.
- No prop drilling, no extra `useState`, no `useEffect` needed.

Example:

```
import { useMutation, useQuery, useQueryClient } from "@tanstack/react-query";
import type { AxiosError } from "axios";
import { toast } from "react-toastify";
import { getItems, createItem, type GetItemsParams } from "./[feature].services";
import type { GetItemsResponse, CreateItemRequest, CreateItemResponse } from
"./[feature].dtos";

const QUERY_KEYS = {
  items: ["getItems"] as const,
  categories: ["getCategories"] as const,
} ;

export const useGetItems = (params?: GetItemsParams) =>
  useQuery<GetItemsResponse, AxiosError>({
    queryKey: [...QUERY_KEYS.items, params],
    queryFn: () => getItems(params),
    enabled: !!params,
  });

export const useCreateItem = () => {
  const queryClient = useQueryClient();
  return useMutation<CreateItemResponse, AxiosError, CreateItemRequest>({
    mutationFn: createItem,
    onSuccess: async () => {
      toast.success("Item created.");
      await queryClient.invalidateQueries({ queryKey: QUERY_KEYS.items });
    },
    onError: () => {
      toast.error("Unable to create item. Please try again.");
    },
  });
};
```

2.5 `helpers/mapItemDtoToRow.ts` — DTO → Row Mapper

Purpose: Pure function. Transforms API shape to table shape. `ItemRow` type is co-located here.

Item	Kind	Description
`ItemRow`	`type`	Flat table row (`id`, `name`, `year`, etc.) — co-located with its mapper
`mapItemDtoToRow(dto)`	`function`	`ItemDto → ItemRow`. Renames PascalCase → camelCase, formats dates with `dayjs`

Rule: `ItemRow` lives with its mapper, not in `types.ts`. Things that change together live together.

Example:

```
import dayjs from "dayjs";
import type { ItemDto } from "../Services/[feature].dtos";

export type ItemRow = {
  id: number;
  name: string;
  year: string;
  lastActivityDate: string | null;
  // ... all table-facing fields
};

export const mapItemDtoToRow = (dto: ItemDto): ItemRow => ({
  id: dto.StudentNumber,
  name: dto.StudentName,
  year: dto.CurrentYear,
  lastActivityDate: dto.LastActivityDate ?
dayjs(dto.LastActivityDate).format("DD/MM/YYYY") : null,
  // ...
});
```

2.6 `helpers/buildEditContext.ts` — DTO → Edit Context

Purpose: Pure function. Builds the pre-populated modal context from a DTO.

Item	Kind	Description
`buildEditContext(dto)`	`function`	`ItemDto → ItemEditContext`. Sets `mode: "update"`, maps fields, defaults missing values

Example:

```
import type { ItemDto } from "../Services/[feature].dtos";
import type { ItemEditContext } from "../Services/[feature].types";

export const buildEditContext = (dto: ItemDto): ItemEditContext => ({
  mode: "update",
  studentActivityId: dto.LastActivityId,
  studentId: dto.StudentNumber,
  studentKey: dto.StudentKey,
  activityId: dto.LastActivityId,
  completedDate: dto.LastActivityDate ? new Date(dto.LastActivityDate) : new Date(),
```

```
  completedBy: 0,
  notes: "",
  restrictedNotes: false,
});
```

2.7 `columns/itemColumns.tsx` — Column Definitions

Purpose: Declarative table config. To add/remove/reorder columns, edit this file only.

Item	Kind	Description
`ActionItem<T>`	`type`	Single row action: `{ icon, label, onClick, variant? }`
`ActionsConfig<T>`	`type`	`{ quick?: ActionItem[], menu?: (ActionItem | "separator")[] }`
`FILTERABLE_COLUMNS`	`const` array	Which columns appear in the filter builder + their type (`string`, `number`, `date`)
`DEFAULT_COLUMN_VISIBILITY`	`const` (`VisibilityState`)	Columns hidden by default (e.g., `homeClass: false`)
`renderActionsCell(row, config)`	`function`	Renders quick buttons + dropdown menu from `ActionsConfig`
`buildItemColumns({ actions? })`	`function`	Returns `ColumnDef<ItemRow>[]`. Conditionally includes actions column via spread

Patterns used:

- `satisfies ColumnDef<ItemRow>` on actions column — compile-time check without type widening.
- Conditional spread `...(actions ? [{ ... } satisfies ColumnDef<ItemRow>] : [])` — actions column only included when config is provided.

Example (key parts):

```
export type ActionsConfig<T> = {
  quick?: ActionItem<T>[];
  menu?: (ActionItem<T> | "separator")[];
};

export const FILTERABLE_COLUMNS: FilterableColumn[] = [
  { id: "name", label: "Name", type: "string", valueType: "text" },
  { id: "year", label: "Year", type: "string", valueType: "dropdown" },
];

export const DEFAULT_COLUMN_VISIBILITY: VisibilityState = {
  homeClass: false,
  coreClass: false,
};

export const buildItemColumns = ({ actions }: { actions?: ActionsConfig<ItemRow> } = {}):
ColumnDef<ItemRow>[] => [
  { id: "select", header: ({ table }) => <Checkbox ... />, cell: ({ row }) => <Checkbox
```

```
  ... />, size: 1 },
    { accessorKey: "name", header: ({ column }) => <TableHeaderButton label="Name" column=
{column} />, size: 14 },
    // ... all columns ...
    ...(actions
      ? [{ id: "actions", cell: ({ row }) => renderActionsCell(row.original, actions) }
satisfies ColumnDef<ItemRow>]
      : []),
  ];
```

2.8 `hooks/use[Feature]Page.ts` — Page Hook

Purpose: All state, handlers, and derived data. The page component just calls this and renders.

Item	Kind	Description
Types		
`ModalState`	`type`	`{ modalOpen, editCtx, deleteConfirmOpen }`
`ModalAction`	`type`	Discriminated union: `OPEN_CREATE | OPEN_EDIT | CLOSE_MODAL | OPEN_DELETE_CONFIRM | CLOSE_DELETE_CONFIRM`
Constants		
`initialModalState`	`const`	Starting state: all closed, `editCtx: null`
`modalReducer(state, action)`	`function`	Pure reducer: `(ModalState, ModalAction) → ModalState`
Inside the hook		
`[modal, dispatch]`	`useReducer`	Coupled modal state (open + context always change together)
`[serverParams, setServerParams]`	`useState`	Server filter params (`undefined` until first search)
`itemsResponse, isLoading, isFetching, isError`	`useGetItems(serverParams)`	List data
`categoriesResponse`	`useGetCategories()`	Dropdown data
`yearsResponse`	`useGetYears()`	Year filter options
`items`	`useMemo`	`itemsResponse ?? []`
`rows`	`useMemo`	`items.map(mapItemDtoToRow)`
`categories`	`useMemo`	`categoriesResponse?.ResponseActivities ?? []`
`yearOptions`	`useMemo`	`yearsResponse?.YearList.map(y => y.Year) ?? []`

Item	Kind	Description
`handleServerSearch(params)`	`useCallback`	`setServerParams(params)`
`handleCreate()`	`useCallback`	`dispatch({ type: "OPEN_CREATE" })`
`handleEdit(itemId)`	`useCallback`	Find DTO → `dispatch({ type: "OPEN_EDIT", ctx: buildEditContext(dto) })`
`handleCloseModal()`	`useCallback`	`dispatch({ type: "CLOSE_MODAL" })`
`handleOpenDeleteConfirm()`	`useCallback`	`dispatch({ type: "OPEN_DELETE_CONFIRM" })`
`handleCloseDeleteConfirm()`	`useCallback`	`dispatch({ type: "CLOSE_DELETE_CONFIRM" })`
Return object		`{ rows, categories, yearOptions, isLoading, isFetching, isError, hasSearched, modalOpen, editCtx, deleteConfirmOpen, handle* }`

Rules:

- `useReducer` for coupled state (modal open + edit context + delete confirm).
- `useState` for independent values (server params).
- `useMemo` for all derived data (prevents re-mapping on every render).
- `useCallback` for all handlers (stabilizes references for child components).

Example (reducer):

```
type ModalAction =
  | { type: "OPEN_CREATE" }
  | { type: "OPEN_EDIT"; ctx: ItemEditContext }
  | { type: "CLOSE_MODAL" }
  | { type: "OPEN_DELETE_CONFIRM" }
  | { type: "CLOSE_DELETE_CONFIRM" };

const modalReducer = (state: ModalState, action: ModalAction): ModalState => {
  switch (action.type) {
    case "OPEN_CREATE":
      return { ...state, modalOpen: true, editCtx: { mode: "create", /* defaults */ } };
    case "OPEN_EDIT":
      return { ...state, modalOpen: true, editCtx: action.ctx };
    case "CLOSE_MODAL":
      return { ...state, modalOpen: false, editCtx: null };
    case "OPEN_DELETE_CONFIRM":
      return { ...state, deleteConfirmOpen: true };
    case "CLOSE_DELETE_CONFIRM":
      return { ...state, deleteConfirmOpen: false };
    default:
      return state;
  }
};
```

2.9 `[Feature].page.tsx` — Orchestrator

Purpose: Thin wiring layer. Calls the hook, renders child components. **No business logic.**

Item	Kind	Description
`use[Feature]Page()`	hook call	Destructures all data + handlers
`[pendingDeleteIds, setPendingDeleteIds]`	`useState<number[]>`	Bridges row-action delete and multi-select delete
`handleDeleteSelected(ids)`	`useCallback`	Sets pending IDs → opens delete confirm
`handleDeleteCompleted()`	`useCallback`	Clears pending IDs after delete
JSX		
`<SingleColumnPage>`	layout	Page wrapper with window title
`<ItemTable>`	component	Receives `rows`, `isLoading`, `isError`, handlers — **error renders inside table, search always visible**
`<ItemModal>`	component	Receives `open`, `editCtx`, `categories` — **conditional render on `editCtx`**
`<DeleteConfirmModal>`	component	Receives `open`, `selectedIds`, callbacks

Rules:

- The page file should be ~80–100 lines max. If it's growing, logic belongs in the hook.
- **Components that can self-fetch MUST NOT receive query result data as props.** If a component calls its own `useQuery` hook, the page must NOT pass that same data down as a prop — that is prop drilling and is forbidden.
- **The page passes only: event handlers, IDs/keys needed to scope a query, and open/close state.** It does NOT pass fetched data objects to components that can fetch their own.

Forbidden anti-patterns on the page:

```
// ❌ WRONG — prop drilling query result into a self-fetching component
const { data: settings } = useSettingsQuery();
return <GreetingBanner nameFormat={settings.nameFormat} />;

// ☑ CORRECT — GreetingBanner calls useSettingsQuery() itself
return <GreetingBanner />;

// ❌ WRONG — passing fetched list data as prop when component can query it
const { data: categories } = useCategoriesQuery();
return <ItemForm categories={categories} />; // if ItemForm can call useCategoriesQuery
itself

// ☑ CORRECT — only pass data the component cannot derive itself (e.g. IDs, handlers)
return <ItemModal open={modalOpen} editCtx={editCtx} onClose={handleClose} />;
```

Example:

```
const [Feature]Page = () => {
  const { rows, categories, yearOptions, isLoading, isFetching, isError, hasSearched,
```

```
    modalOpen, editCtx, deleteConfirmOpen,
    handleServerSearch, handleCreate, handleEdit, handleCloseModal,
    handleOpenDeleteConfirm, handleCloseDeleteConfirm,
  } = use[Feature]Page();

  const [pendingDeleteIds, setPendingDeleteIds] = useState<number[]>([]);

  const handleDeleteSelected = useCallback((ids: number[]) => {
    setPendingDeleteIds(ids);
    handleOpenDeleteConfirm();
  }, [handleOpenDeleteConfirm]);

  const handleDeleteCompleted = useCallback(() => {
    setPendingDeleteIds([]);
  }, []);

  return (
    <SingleColumnPage windowTitle="[Feature]: Items">
      <PageTitle>[Feature]: Items</PageTitle>
      <PageContent>
        {/* Table — error renders inside, search filters always visible */}
        <ItemTable rows={rows} isLoading={isLoading} isFetching={isFetching} isError=
{isError} hasSearched={hasSearched}
          yearOptions={yearOptions} onServerSearch={handleServerSearch}
          onCreate={handleCreate} onEdit={handleEdit} onDeleteSelected=
{handleDeleteSelected} />
        {editCtx && <ItemModal open={modalOpen} onClose={handleCloseModal} editCtx=
{editCtx} categories={categories} />}
        <DeleteConfirmModal open={deleteConfirmOpen} onClose={handleCloseDeleteConfirm}
          selectedIds={pendingDeleteIds} onDeleted={handleDeleteCompleted} />
      </PageContent>
    </SingleColumnPage>
  );
};
```

2.10 `components/ItemTable.tsx` — Data Table

Purpose: Renders the table with search, filters, pagination, row selection, skeleton loading.

Item	Kind	Description
`PAGE_SIZE`	`const`	`10`
`SKELETON_COLS`, `SKELETON_ROWS`, `SKELETON_WIDTHS`	`const`	Skeleton loading config
`TableSkeletonRows()`	`function component`	Renders skeleton loading rows
`ItemTableProps`	`type`	`{ rows, isLoading, isFetching, isError, hasSearched, yearOptions, onServerSearch, onCreate, onEdit, onDeleteSelected }`
Inside the component		

Item	Kind	Description
`[sorting, setSorting]`	`useState<SortingState>`	Column sort state
`[searchValue, setSearchValue]`	`useState<string>`	Client-side text filter
`[columnFilters, setColumnFilters]`	`useState<ColumnFiltersState>`	Filter builder state
`[rowSelection, setRowSelection]`	`useState<Record<string, boolean>>`	Checked rows
`[selectedYear, setSelectedYear]`	`useState<string>`	Server filter dropdown
`actions`	`useMemo<ActionsConfig>`	Row action buttons config (depends on `onEdit`, `onDeleteSelected`)
`columns`	`useMemo`	`buildItemColumns({ actions })` (depends on `actions`)
`table`	`useReactTable(...)`	Table instance with 4 row models + custom `filterBuilder` filterFn
`selectedIds`	`useMemo`	Derives checked row IDs from `table.getSelectedRowModel()`
`selectedCount`	derived	`selectedIds.length`
`totalSize`, `getColWidth(size)`	derived	Percentage-based column widths for `table-fixed` layout
`pageCount`, `currentPage`	derived	Pagination values
JSX slots		
`<TableServerFilters>`	slot	Server-side filter dropdowns + Search button
`<TableSearch>`	slot	Client-side text filter input
`<TablePrimaryFilters>`	slot	`<TableFilterBuilder>` component
`<TablePrimaryButtons>`	slot	Delete (count) + Create Item buttons
`<TableContent>`	slot	**Error** → **Skeleton** → **empty** → **table** with `flexRender`. Error renders here (not at page level) so search filters stay visible
`<TablePaginationMobileFriendly>`	slot	Page navigation (shown when `pageCount > 1`)

Rules:

- `useState` for each independent table UI state (sorting, search, filters, selection).
- `useMemo` chain: `actions` → `columns` (prevents rebuilding on unrelated state changes).
- `useReactTable` with `getCoreRowModel`, `getSortedRowModel`, `getFilteredRowModel`, `getPaginationRowModel`.

- Custom `filterBuilder` filterFn registered via `filterFns` + `"filterBuilder" as never` bridge.
- Column widths use `size` as proportions, normalized to percentages via `getColWidth`.

2.11 `components/ItemModal.tsx` — Create/Edit Dialog

Purpose: Wraps `ItemForm` in a Dialog with `FormProvider`. Handles submit logic.

Item	Kind	Description
`ItemModalProps`	`type`	`{ open, onClose, editCtx, categories, categoriesLoading? }`
`buildDefaults(ctx)`	`function` (outside component)	`ItemEditContext → ItemFormValues` (pure)
`buildPayload(values)`	`function` (outside component)	`ItemFormValues → mutation payload` (pure)
Inside the component		
`isEdit`	derived	`editCtx.mode === "update"`
`triggerRef`	`useRef<HTMLElement>`	Stores the button that opened the modal (WCAG focus return)
`createMutation`	`useCreateItem()`	Create mutation hook
`updateMutation`	`useUpdateItem()`	Update mutation hook
`defaultValues`	`useMemo`	`buildDefaults(editCtx)`
`methods`	`useForm<ItemFormValues>`	Form instance with `zodResolver(ItemFormSchema)`
`handleSubmit`, `reset`	destructured from `methods`	Submit wrapper + form reset
`useEffect` #1	effect	Captures `document.activeElement` into `triggerRef` when modal opens
`useEffect` #2	effect	Calls `reset(defaultValues)` when modal opens or context changes
`onSubmit(values)`	`useCallback`	Builds payload → calls `createMutation.mutate` or `updateMutation.mutate`
`isSubmitting`	derived	`createMutation.isPending || updateMutation.isPending`
JSX		
`<Dialog>`	wrapper	Controlled by `open` prop
`<DialogContent onCloseAutoFocus>`	content	Returns focus via `triggerRef.current?.focus()`
`<FormProvider {...methods}>`	context	Shares form to `<ItemForm>`
`<form onSubmit={handleSubmit(onSubmit)}>`	form	Validation gate — blocks submit if Zod fails

Item	Kind	Description
`<ItemForm>`	child	Renders the fields
`<DialogFooter>`	footer	Cancel + Submit buttons (disabled while `isSubmitting`)

Rules:

- `buildDefaults` and `buildPayload` are pure functions **outside** the component (testable, no hooks).
- `useEffect` for reset — because `useForm` only reads `defaultValues` on first mount.
- `useRef` for focus return — WCAG requirement, no re-render needed.
- `FormProvider` wraps the form so `ItemForm` can use `useFormContext`.

2.12 `components/ItemForm.tsx` — Form Fields

Purpose: Renders fields. Reads form state from `FormProvider` via `useFormContext`. **No submit logic.**

Item	Kind	Description
`ItemFormProps`	`type`	`{ categories, categoriesLoading?, isSubmitting?, completedByCode? }`
Inside the component		
`[dateSelectorOpen, setDateSelectorOpen]`	`useState<boolean>`	Calendar popover open/close
`loggedInUserCode`	`useMemo` (empty deps)	Reads `localStorage` once, cached forever
`{ control, watch, formState: { errors, submitCount } }`	`useFormContext<ItemFormValues>()`	Reads from parent `FormProvider`
`notes`	`watch("notes")`	Live subscription to notes field value
`showErrorMessage`	derived	`submitCount > 0 && Object.keys(errors).length > 0`
JSX fields (each wrapped in `<Controller>`)		
Activity	`<Select>`	Dropdown of categories
Completed By	`<TeacherSearch>`	Staff autocomplete
Completed Date	`<Popover>` + `<Calendar>`	Date picker
Notes	`<Textarea>`	Free text
Restrict Notes	`<Checkbox>`	**Conditional** — only shown when `!!notes` (via `watch`)

Item	Kind	Description
Error Summary	`<Alert>`	Shown after first submit attempt if errors exist

Rules:

- Every non-native input (Shadcn Select, Checkbox, Calendar, etc.) is wrapped in `<Controller>`.
- `useFormContext` — no prop drilling of form methods.
- `watch("fieldName")` for conditional rendering based on field values.
- Error messages use `aria-invalid` + `aria-describedby` for WCAG.
- All inputs get `disabled={isSubmitting}` during save.

2.13 `components/DeleteConfirmModal.tsx` — Delete Confirmation

Purpose: Confirms multi-select delete with partial failure handling.

Item	Kind	Description
`DeleteConfirmModalProps`	`type`	`{ open, onClose, selectedIds, onDeleted }`
Inside the component		
`[isDeleting, setIsDeleting]`	`useState<boolean>`	Local loading state
`deleteMutation`	`useDeleteItem({ onDeleted })`	Delete mutation with callback
`count`	derived	`selectedIds.length`
`label`	derived	`"Item"` or `"Items"` (singular/plural)
`handleConfirm()`	`useCallback` (async)	`Promise.allSettled` over all IDs → counts succeeded/failed → appropriate toast
JSX		
`<AlertDialog>`	wrapper	Controlled by `open`
Cancel button	`<AlertDialogCancel>`	Disabled while deleting
Confirm button	`<AlertDialogAction>`	Shows "Deleting..." while in progress, destructive styling

Rules:

- `Promise.allSettled` (not `Promise.all`) — handles partial failures gracefully.
- Three toast outcomes: all succeeded, all failed, partial (X deleted, Y failed).
- `AlertDialog` (not `Dialog`) — semantically correct for destructive confirmations.

2.14 `components/StatusPlaceholder.tsx` — Status Display

Purpose: Reusable placeholder for empty/error states.

Item	Kind	Description
`StatusPlaceholderProps`	`type`	`{ children, variant?: "muted" | "error" }`

Item	Kind	Description
Component	functional	`<div role="status" aria-live="polite">` with variant-based styling

Rule: `aria-live="polite"` ensures screen readers announce state changes (WCAG).

2.15 `[Feature].module.tsx` — Route Entry Point

Purpose: Code splitting via `lazy` + `Suspense`.

Item	Kind	Description
`[Feature]Page`	`lazy(() => import(...))`	Dynamically imported page component
`[Feature]Module`	component	`<Suspense fallback>` → `<Routes>` → `<Route index element={<Page />} />`

Rule: Every feature gets a Module file. The router imports the Module, not the Page directly.

Example:

```
import { lazy, Suspense } from "react";
import { Route, Routes } from "react-router-dom";

const [Feature]Page = lazy(() => import("./[Feature].page"));

const [Feature]Module = () => (
  <Suspense fallback={<div aria-live="polite">Loading...</div>}>
    <Routes>
      <Route index element={<[Feature]Page />} />
    </Routes>
  </Suspense>
);

export default [Feature]Module;
```

---

3. Data Flow Diagram

```
[Feature].module.tsx
  └── lazy loads → [Feature].page.tsx (orchestrator)
        │
        ├── calls → use[Feature]Page.ts (hook)
        │     ├── useReducer     → modal state (open/close/edit context)
        │     ├── useState       → serverParams
        │     ├── useGetItems    → itemsResponse  ──→ useMemo → items → rows
        │     ├── useGetCategories → categoriesResponse ──→ useMemo → categories
        │     ├── useGetYears    → yearsResponse ──→ useMemo → yearOptions
        │     └── useCallback    → all handle* functions
        │
        ├── renders → <ItemTable>
```

```
│     ├── useState × 5   → sorting, search, filters, selection, serverFilter
│     ├── useMemo        → actions config → columns → selectedIds
│     ├── useReactTable  → table instance
│     └── flexRender     → renders cells
│
├── renders → <ItemModal>
│     ├── useForm        → form instance (zodResolver)
│     ├── useMemo        → defaultValues
│     ├── useRef         → triggerRef (WCAG focus return)
│     ├── useEffect × 2  → capture focus + reset form
│     ├── useCallback    → onSubmit
│     ├── FormProvider   → shares form to ↓
│     └── <ItemForm>
│           ├── useFormContext → reads control, watch, errors
│           ├── watch("notes") → conditional checkbox
│           └── Controller × N → bridges Shadcn inputs
│
└── renders → <DeleteConfirmModal>
      ├── useState       → isDeleting
      ├── useDeleteItem  → mutation
      └── useCallback    → handleConfirm (Promise.allSettled)
```

---

4. React Hooks & Patterns — Where Each Is Used

Hook / Pattern	Files	Purpose
`useState`	Page, ItemTable, ItemForm, DeleteConfirmModal	Independent UI state
`useReducer`	use[Feature]Page	Coupled modal state (open + context)
`useMemo`	use[Feature]Page, ItemTable, ItemModal, ItemForm	Cache derived data + stabilize references
`useCallback`	use[Feature]Page, Page, ItemModal, DeleteConfirmModal	Stabilize handler references for children
`useEffect`	ItemModal (×2)	Focus capture + form reset (side effects)
`useRef`	ItemModal	WCAG focus return (no re-render needed)
`lazy` + `Suspense`	Module	Code splitting
`useForm`	ItemModal	Form instance with Zod validation
`FormProvider`	ItemModal	Shares form context to ItemForm
`useFormContext`	ItemForm	Reads form state from parent
`Controller`	ItemForm (×N)	Bridges RHF with Shadcn components
`watch`	ItemForm	Conditional rendering based on field value
`zodResolver`	ItemModal	Wires Zod schema into RHF

Hook / Pattern	Files	Purpose
`useQuery`	queries (×3)	Declarative GET with caching
`useMutation`	queries (×3)	Declarative POST with side effects
`useQueryClient`	queries (×3)	Cache invalidation after mutations
`useReactTable`	ItemTable	Headless table instance
`flexRender`	ItemTable	Renders column defs to JSX
`z.object` / `z.infer`	dtos, types	Schema + inferred type (single source of truth)
`zodParse`	services	Runtime API response validation
`Promise.allSettled`	DeleteConfirmModal	Partial failure handling in multi-delete
`satisfies`	itemColumns	Compile-time type check without widening
`as const`	queries	Readonly literal query keys

5. Rules & Conventions

File Naming

- Services: `[feature].dtos.ts`, `[feature].types.ts`, `[feature].services.ts`, `[feature].queries.ts`
- Helpers: `map[Entity]DtoToRow.ts`, `buildEditContext.ts`
- Columns: `[entity]Columns.tsx`
- Hook: `use[Feature]Page.ts`
- Components: PascalCase (`ItemTable.tsx`, `ItemModal.tsx`, `ItemForm.tsx`, `DeleteConfirmModal.tsx`)
- Page: `[Feature].page.tsx`
- Module: `[Feature].module.tsx`

Separation of Concerns

- **Services/** — no React, no UI. Pure API + types.
- **helpers/** — no React, no side effects. Pure functions.
- **columns/** — declarative config only. No state, no hooks.
- **hooks/** — all state + handlers. No JSX.
- **components/** — rendering only. Minimal local state (UI-only like sort/search). **Each component owns its own data fetching via `useQuery` hooks — no query results are passed in as props from the page.**
- **Page** — thin orchestrator. Calls hook, renders components. No business logic. Passes only: handlers, IDs/keys, and open/close state.
- **Module** — code splitting only.

Component Data Ownership (MANDATORY)

This is one of the most important rules in the blueprint:

> **A component that calls a `useQuery` hook must NOT also receive that same data as a prop from its parent.**

React Query's cache means every component calling the same query key gets the same cached data — there is no performance cost to calling `useQuery` in multiple components. Prop drilling query results is therefore always wrong.

Decision tree — should a component receive data as a prop or fetch it?

Scenario	Pattern
Component needs data from an API	Call `useQuery` inside the component
Component needs a scoping ID (e.g. `studentId`)	Receive as prop — it cannot derive this itself
Component needs an event handler	Receive as prop — handlers come from the hook
Component needs open/close state	Receive as prop — UI state lives in the page hook
Component needs API data that the page also uses	Both call the same query — React Query deduplicates

URL Param Overrides (MANDATORY pattern)

When query results need to be overridable via URL search params (e.g. for testing, embedding, or feature flags):

1. **Override in the query hook** using React Query's `select` option — NOT in the page, NOT as props.
2. **Parse helpers are pure functions** defined outside the hook (no React, fully testable).
3. **Priority chain:** URL param → API result → hardcoded default.
4. **Invalid param values** are silently ignored and fall through to the next level.

```
// ☑ Correct — override at the query boundary
export const useSettingsQuery = () => {
  const [searchParams] = useSearchParams();
  return useQuery({
    queryKey: QUERY_KEYS.settings,
    queryFn: Services.getSettings,
    select: (data) => ({
      showWidget: parseBoolParam(searchParams.get("showWidget")) ?? data.showWidget,
      nameFormat: parseEnumParam(searchParams.get("nameFormat"), VALID_FORMATS) ??
data.nameFormat,
    }),
  });
};

// ❌ Wrong — override at the page level and prop-drill
const { data: settings } = useSettingsQuery();
const nameFormat = searchParams.get("nameFormat") ?? settings.nameFormat;
return <GreetingBanner nameFormat={nameFormat} />; // ← prop drilling
```

Performance

- `useMemo` for all `.map()`, `.filter()`, objects/arrays passed as props.
- `useCallback` for all handlers passed as props to children.
- `useMemo` chain: `actions` → `columns` (prevents cascading rebuilds).
- `useReducer` for coupled state (prevents out-of-sync bugs).

Forms

- One `useForm` per modal, with `zodResolver`.
- `FormProvider` in the modal, `useFormContext` in the form.
- Every Shadcn input wrapped in `<Controller>`.
- `watch` for conditional rendering.

- `reset()` in `useEffect` for re-opening with new data.

Accessibility (WCAG)

- `useRef` + `onCloseAutoFocus` for focus return on modal close.
- `aria-invalid` + `aria-describedby` on form fields with errors.
- `aria-live="polite"` on status placeholders.
- `aria-label` on icon-only buttons.
- `AlertDialog` for destructive confirmations.

Error Handling

- `zodParse` validates API responses at the boundary.
- `Promise.allSettled` for multi-delete (partial failure handling).
- Toast notifications for all mutation outcomes (success/error).
- `StatusPlaceholder` for page-level error states.

TypeScript

- Types inferred from Zod schemas (`z.infer`) — never manually duplicated.
- `as const` for query keys.
- `satisfies` for compile-time checks without widening.
- Discriminated unions for reducer actions.

Shared UI Components (use these — do not reinvent)

Component	Location	Use when
`TablePaginationMobileFriendly`	`@/components/layouts/v1/Table.layout.tsx`	All paginated tables (replaces `TablePagination`)
`TableFilterBuilder`	`@/components/layouts/v1/TableFilterBuilder.tsx`	Dynamic client-side column filtering
`TableServerFilters`	`@/components/layouts/v1/Table.layout.tsx`	Server-side filter dropdowns + Search button
`TableSearch`	`@/components/layouts/v1/Table.layout.tsx`	Client-side text search input
`TablePrimaryButtons`	`@/components/layouts/v1/Table.layout.tsx`	Action buttons (Create, Delete count)
`StatusPlaceholder`	feature `components/`	Empty / error state display

`TablePaginationMobileFriendly` API:

```
<TablePaginationMobileFriendly
  pageCount={pageCount}
  currentPage={currentPage}
```

```
  onPageChange={(p) => table.setPageIndex(p - 1)}
  totalItems={totalItems} // optional — shows "Total of X items"
/>
```

`TableFilterBuilder` **usage (3 steps):**

1. Define `FILTERABLE_COLUMNS: FilterableColumn[]` in `itemColumns.tsx`
2. Drop `<TableFilterBuilder columns={...} filters={columnFilters} onFiltersChange={setColumnFilters} rows={rows} />` into the table
3. Register on TanStack Table: `filterFns: { filterBuilder: (row, id, val) => filterBuilderFn(row, id, val, FILTERABLE_COLUMNS) }` + `defaultColumn: { filterFn: "filterBuilder" }`

Deviation Rule

> **Before implementing anything that deviates from this blueprint, you MUST:**
>
> 1. Identify the specific rule being broken.
> 2. Explain why the deviation is being considered.
> 3. Get explicit approval from the team/user.
> 4. Document the approved deviation.
>
> **Silent deviations are never acceptable**, even if the alternative seems simpler or faster.

---

6. Architecture Principles

Vertical Slice Architecture (VSA)

Each feature is a **self-contained vertical slice** — it owns everything it needs from API contract to UI rendering. No feature reaches into another feature's internals.

```
src/features/
├── staff-absence/       ← complete vertical slice
│   ├── Services/        ← API layer for THIS feature only
│   ├── helpers/         ← pure functions for THIS feature only
│   ├── columns/         ← table config for THIS feature only
│   ├── hooks/           ← state for THIS feature only
│   └── components/      ← UI for THIS feature only
├── subject-classifications/  ← another complete vertical slice
└── time-room/               ← another complete vertical slice
```

Rules:

- Features NEVER import from each other's `Services/`, `helpers/`, `hooks/`, or `components/`.
- Shared code lives in `src/components/` (UI primitives) or `src/hooks/` (global hooks like `useAuth`).
- If two features need the same logic, extract it to `src/` — never copy-paste between features.

Clean Architecture (Dependency Direction)

Dependencies always point **inward** — outer layers depend on inner layers, never the reverse.

```
UI Components          ← outermost (knows about everything below)
    ↓
Page / Hook            ← orchestration layer
    ↓
React Query Hooks      ← data access layer
    ↓
Services               ← HTTP layer (no React, no UI)
    ↓
DTOs / Types           ← innermost (pure data shapes, no dependencies)
```

Rule: `dtos.ts` and `types.ts` import nothing from this project. `services.ts` imports only from `dtos.ts`. `queries.ts` imports from `services.ts` and `dtos.ts`. Components import from queries and types. **Never reverse this direction.**

Screaming Architecture

The folder structure **screams what the application does**, not how it is built.

```
☑ src/features/staff-absence/     ← screams "staff absence management"
☑ src/features/subject-classifications/  ← screams "subject classification CRUD"
❌ src/pages/form/                  ← says nothing about the domain
❌ src/containers/modal/            ← says nothing about the domain
```

Rule: Every folder name must be a **domain concept**, not a technical concept. A new developer reading the folder tree should immediately understand what the app does.

DRY (Don't Repeat Yourself)

- One Zod schema → one inferred type. Never define the same shape twice.
- One `QUERY_KEYS` constant → used everywhere for that feature's cache keys.
- One `buildItemColumns` factory → used by the table, never duplicated.
- Shared UI primitives in `src/components/` — never copy Shadcn components into features.

KISS (Keep It Simple, Stupid)

- If a function is more than ~20 lines, ask: can it be split?
- If a component is more than ~150 lines, ask: can a sub-component be extracted?
- If a hook is more than ~100 lines, ask: is it doing too much?
- Prefer explicit over clever. A junior developer must be able to read it in 30 seconds.

Simple beats smart. Readable beats terse.

---

7. Security — OWASP Standards

A01 — Broken Access Control

- **Never** render UI based on client-side role checks alone. The backend enforces access.
- Use `useAuth()` for UI hints only (show/hide buttons) — never as a security gate.
- All API calls go through `apiClient` which attaches the JWT automatically.

```
// ☑ Correct — UI hint only, backend still enforces
const { user } = useAuth();
const isAdmin = user?.IsSuperUser === "True";
return isAdmin ? <DeleteButton /> : null;  // backend will reject if not actually admin
```

A02 — Cryptographic Failures

- JWT tokens are stored in `localStorage` and attached via `apiClient` headers — never hardcoded.
- Never log tokens, passwords, or sensitive data to the console.
- Never store sensitive data in component state beyond what is needed for the current render.

A03 — Injection

- **All user input goes through Zod validation** before being sent to the API.
- `zodResolver` on every form — invalid data never reaches `mutationFn`.
- Never use `dangerouslySetInnerHTML`. If rich text is needed, use a sanitised renderer.

```
// ☑ Correct — Zod validates before mutation fires
export const ItemFormSchema = z.object({
  code: z.string().min(1).max(10),  // length-bounded, prevents oversized payloads
  description: z.string().min(1).max(255),
});
```

A04 — Insecure Design

- Forms use `disabled={isSubmitting}` to prevent double-submit.
- Delete actions always require a confirmation dialog (`AlertDialog`) — no one-click deletes.
- Destructive operations use `Promise.allSettled` — partial failures are reported, not silently swallowed.

A05 — Security Misconfiguration

- No API keys, secrets, or environment-specific URLs hardcoded in source files.
- Base URLs come from `appSettings` or environment config, not string literals in components.

A06 — Vulnerable Components

- Use only packages from `package.json`. No CDN imports, no inline scripts.
- Keep dependencies updated. Flag outdated packages in code reviews.

A07 — Identification & Authentication Failures

- `apiClient` attaches JWT on every request — no manual token handling in features.
- Expired token handling is centralised in `apiClient` interceptors — not scattered across features.

A08 — Software & Data Integrity Failures

- `zodParse` validates **all** API responses at the boundary — malformed data never reaches the UI.
- Never skip `zodParse` on read endpoints, even for "simple" responses.

```
// ☑ Always validate — even simple responses
export const getItems = async (): Promise<GetItemsResponse> => {
```

```
  const response = await apiClient.get("/api/items");
  return zodParse(GetItemsResponseSchema, response.data, "getItems"); // ← mandatory
};
```

A09 — Logging & Monitoring Failures

- Use `zodParse` with a descriptive label — parse failures are logged with context.
- Toast errors give users actionable messages, not raw error objects.
- Never expose stack traces or internal error details in toast messages.

A10 — Server-Side Request Forgery

- All API calls go through `apiClient` with a known base URL — no user-controlled URLs.
- Never construct API URLs from user input.

---

8. Accessibility — WCAG 2.1 AA

Perceivable

- **Text alternatives:** All icon-only buttons have `aria-label`.
- **Colour contrast:** Use Shadcn's design tokens — they meet 4.5:1 contrast ratio by default.
- **Status messages:** `aria-live="polite"` on `StatusPlaceholder` and Suspense fallbacks.
- **Error messages:** Linked to inputs via `aria-describedby` — screen readers announce them.

```
// ☑ Every icon-only button
<Button variant="ghost" size="icon-sm" aria-label="Edit item">
  <EditIcon />
</Button>

// ☑ Every form field with an error
<Input
  aria-invalid={!!errors.code}
  aria-describedby={errors.code ? "code-error" : undefined}
/>
{errors.code && <p id="code-error">{errors.code.message}</p>}
```

Operable

- **Keyboard navigation:** All interactive elements are reachable via Tab. Modals trap focus.
- **Focus return:** `useRef` + `onCloseAutoFocus` returns focus to the trigger element when a modal closes.
- **No keyboard traps:** `DialogContent` from Shadcn handles this automatically.
- **Skip links:** Not required for feature pages (handled at the app shell level).

```
// ☑ Focus return on modal close — mandatory in every modal
const triggerRef = useRef<HTMLElement | null>(null);

useEffect(() => {
  if (open) triggerRef.current = document.activeElement as HTMLElement;
}, [open]);

<DialogContent onCloseAutoFocus={() => triggerRef.current?.focus()}>
```

Understandable

- **Labels:** Every form field has a visible `<Label>` linked via `htmlFor` / `id`.
- **Error messages:** Written in plain language. "Code is required" not "Validation failed".
- **Loading states:** Skeleton rows during load — not blank screens.
- **Confirmation dialogs:** `AlertDialog` for destructive actions — clear title + description.

Robust

- **Semantic HTML:** Use `<button>` not `<div onClick>`. Use `<form>` not `<div onSubmit>`.
- **ARIA roles:** `role="status"` on status placeholders. `AlertDialog` uses `role="alertdialog"` automatically.
- **No `tabIndex > 0`:** Never use positive tabindex — it breaks natural tab order.

WCAG Checklist (verify before shipping)

- ☐ All icon-only buttons have `aria-label`
- ☐ All form fields have `<Label htmlFor>` linked to input `id`
- ☐ All form errors have `aria-invalid` + `aria-describedby`
- ☐ All modals return focus on close (`onCloseAutoFocus`)
- ☐ All status/loading areas have `aria-live`
- ☐ Delete confirmations use `<AlertDialog>` not `<Dialog>`
- ☐ No `dangerouslySetInnerHTML`
- ☐ Keyboard-navigable without a mouse

---

9. Mobile Responsiveness

Mandatory Rules

- **All layouts use Tailwind responsive prefixes** (`sm:`, `md:`, `lg:`). No fixed pixel widths.
- **Tables are horizontally scrollable** on small screens — wrap in `overflow-x-auto`.
- **Modals are full-height safe** — use `max-h-dvh overflow-y-auto` on `DialogContent`.
- **Buttons stack vertically** on mobile — use `flex-col sm:flex-row` for button groups.
- **Pagination uses `TablePaginationMobileFriendly`** — compact layout works on 320px screens.

Table on Mobile

```
// ☑ Table wrapper — always scrollable on mobile
<div className="overflow-x-auto rounded-md border">
  <table className="table-fixed w-full min-w-[600px]">
    {/* min-w prevents columns from collapsing below readable width */}
  </table>
</div>
```

Modal on Mobile

```
// ☑ Modal — safe on all screen heights including short mobile screens
<DialogContent className="max-h-dvh overflow-y-auto">
```

Form Layout on Mobile

```
// ☑ Form fields — full width on mobile, constrained on desktop
<div className="flex w-full max-w-110 flex-col gap-4">
```

Button Groups on Mobile

```
// ☑ Buttons — stack on mobile, row on desktop
<div className="flex flex-col gap-2 sm:flex-row sm:justify-end">
  <Button variant="outline">Cancel</Button>
  <Button type="submit">Save</Button>
</div>
```

Shadcn Components — Always Use These

Never build custom UI primitives. Always use Shadcn:

Need	Shadcn Component
Button	`Button` from `@/components/ui/v1/button`
Text input	`Input` from `@/components/ui/v1/input`
Dropdown	`Select` + `SelectContent` + `SelectItem`
Checkbox	`Checkbox` from `@/components/ui/v1/checkbox`
Date picker	`Popover` + `Calendar`
Modal (standard)	`Dialog` + `DialogContent`
Modal (destructive)	`AlertDialog` + `AlertDialogAction`
Toast	`toast.success` / `toast.error` from `react-toastify`
Alert/banner	`Alert` + `AlertDescription`
Label	`Label` from `@/components/ui/v1/label`

---

10. Production Readiness Scorecard

Use this to evaluate any feature before merging. Target: **100%**.

Category 1 — Architecture & Structure (20 points)

Check	Points	Pass criteria
Folder follows VSA structure	4	All 7 folders present: `Services/`, `helpers/`, `columns/`, `hooks/`, `components/`, module, page
Screaming architecture	2	Folder name is a domain concept, not a technical one
No cross-feature imports	4	`grep` for `../other-feature/` returns nothing

Check	Points	Pass criteria
Clean dependency direction	4	`dtos.ts` imports nothing from this project; services import only dtos
Page is thin orchestrator	4	Page file ≤ 100 lines; no business logic; no `useQuery` calls
No prop drilling of query results	2	Components that can self-fetch do not receive API data as props

Category 2 — TypeScript & Data Safety (20 points)

Check	Points	Pass criteria
All response types inferred from Zod	4	No manually duplicated types that mirror a Zod schema
`zodParse` on every read endpoint	6	Every `apiClient.get/post` in services that returns data uses `zodParse`
`z.infer` for form values	4	`ItemFormValues = z.infer<typeof ItemFormSchema>` — never manually defined
`as const` on query keys	2	`QUERY_KEYS` uses `as const`
`satisfies` on actions column	2	Actions column uses `satisfies ColumnDef<ItemRow>`
No `any` types	2	`grep` for `: any` or `as any` returns nothing (except `z.any()` in DTOs where backend is untyped)

Category 3 — React Patterns & Performance (20 points)

Check	Points	Pass criteria
`useReducer` for coupled state	4	Modal open + editCtx + deleteConfirmOpen managed by reducer
`useMemo` for all derived data	4	Every `.map()`, `.filter()`, object/array passed as prop is memoised
`useCallback` for all handlers	4	Every `handle*` function is wrapped in `useCallback`
`useMemo` chain: actions → columns	2	`actions` memo feeds `columns` memo — not rebuilt on unrelated state
`lazy` + `Suspense` in module	2	Module file uses `lazy(() => import(...))`
No unnecessary `useEffect`	4	Effects only for: focus capture, form reset, external subscriptions

Category 4 — Security / OWASP (15 points)

Check	Points	Pass criteria
No hardcoded secrets or URLs	3	No API keys, tokens, or base URLs in source
All form input validated by Zod	4	`zodResolver(ItemFormSchema)` on every `useForm`
No `dangerouslySetInnerHTML`	3	`grep` returns nothing
Double-submit prevented	2	All submit buttons have `disabled={isSubmitting}`

Check	Points	Pass criteria
Delete requires confirmation	3	All deletes go through `AlertDialog` — no one-click deletes

Category 5 — Accessibility / WCAG (15 points)

Check	Points	Pass criteria
All icon-only buttons have `aria-label`	3	`grep` for `size="icon"` — every result has `aria-label`
All form fields have `<Label htmlFor>`	3	Every input has a linked visible label
All form errors have `aria-invalid` + `aria-describedby`	3	Every `Controller` render includes both attributes
Focus returns on modal close	3	Every modal has `triggerRef` + `onCloseAutoFocus`
Status areas have `aria-live`	3	`StatusPlaceholder` and Suspense fallback have `aria-live="polite"`

Category 6 — UX & Mobile (10 points)

Check	Points	Pass criteria
Table is mobile-scrollable	2	Table wrapper has `overflow-x-auto`
Modal is mobile-safe	2	`DialogContent` has `max-h-dvh overflow-y-auto`
Pagination uses mobile-friendly component	2	`TablePaginationMobileFriendly` used (not `TablePagination`)
Skeleton loading (not blank screen)	2	Table shows skeleton rows while `isLoading`
Only Shadcn components used	2	No custom-built buttons, inputs, or modals

Scoring

Score	Grade	Verdict
100	☑ **Production Ready**	Ship it
90–99	⚠ **Near Ready**	Fix gaps before merge
75–89	◇ **Needs Work**	Significant gaps — do not merge
< 75	✗ **Not Ready**	Major rework required

> **Target is 100%. Anything less requires a documented reason for each gap.**

---

11. Junior Developer Quick-Start Guide

"I need to build a new feature. Where do I start?"

1. **Copy the `react-crud-v4` folder** and rename it to your feature name (e.g. `student-awards`).
2. **Rename all files** — replace `crud`/`Item`/`item` with your entity name.
3. **Update `dtos.ts`** — replace the Zod schemas with your API's actual response shapes.
4. **Update `services.ts`** — replace the endpoint URLs with your real API endpoints.

5. **Update** `types.ts` — replace `ItemEditContext` and `ItemFormSchema` with your form fields.
6. **Update** `helpers/mapItemDtoToRow.ts` — map your DTO fields to table row fields.
7. **Update** `columns/itemColumns.tsx` — define your table columns.
8. **Run TypeScript** — `npx tsc --noEmit` — fix all errors before writing any JSX.
9. **Wire the module** into the router.

"What file do I edit to...?"

Task	File to edit
Change what columns appear in the table	`columns/itemColumns.tsx`
Add a new API endpoint	`Services/[feature].services.ts` + `Services/[feature].queries.ts`
Add a new form field	`Services/[feature].types.ts` (schema) + `components/ItemForm.tsx` (UI)
Change what data the table shows	`helpers/mapItemDtoToRow.ts`
Add a new button/action to a row	`columns/itemColumns.tsx` (actions config)
Change modal title or layout	`components/ItemModal.tsx`
Change page layout	`[Feature].page.tsx`
Change loading/error message	`components/StatusPlaceholder.tsx`

"What are the most common mistakes to avoid?"

1. **Don't fetch data in the page** — the page only calls `useFeaturePage()` and renders.
2. **Don't pass API data as props** — if a component needs API data, it calls `useQuery` itself.
3. **Don't skip** `zodParse` — every API response must be validated at the boundary.
4. **Don't write** `type X = { ... }` **if a Zod schema already defines the same shape** — use `z.infer`.
5. **Don't use** `useEffect` **for data fetching** — use `useQuery` instead.
6. **Don't use** `useState` **for coupled state** — use `useReducer` when two values always change together.
7. **Don't build custom UI components** — use Shadcn. It's already there.
8. **Don't forget** `aria-label` **on icon-only buttons** — screen readers need it.
9. **Don't use** `Promise.all` **for multi-delete** — use `Promise.allSettled` so partial failures are handled.
10. **Don't deviate from this blueprint without asking first.**

"How do I know if my code is good?"

Run through the **Production Readiness Scorecard** (Section 10). If you score 100, it's good. If not, fix the gaps. The scorecard is not optional — it is the definition of done.